

# Basics of Python LO1

14-1-26

Aim – Use print, input and arithmetic functions in python to calculate allowances and gross salary as well as calculate simple interest using principal rate of interest and time period.

Theory –

It uses input to get numeric values from the user, and uses type conversion to ensure the data type.

It uses print function to display the calculated result.

We use addition, multiplication and division to calculate the required results.

It helps us show how python can be applied for financial and salary based applications.

A1.

```
print("Please enter your basic salary: ", end="")

basic_salary = float(input())

DA = basic_salary * 0.70

TA = basic_salary * 0.30

HRA = basic_salary * 0.10

print("Dearness Allowance (DA): " + str(DA))

print("Travel Allowance (TA): " + str(TA))

print("House Rent Allowance (HRA): " + str(HRA))

print("Gross Salary: " + str(basic_salary + DA + TA + HRA))
```

```
Please enter your basic salary: 1000
Dearness Allowance (DA): 700.0
Travel Allowance (TA): 300.0
House Rent Allowance (HRA): 100.0
Gross Salary: 2100.0
```

A2.

```
print("Enter principal amount:", end=" ")

P = float(input())

print("Enter rate of interest (in %):", end=" ")

R = float(input())

print("Enter time period (in years):", end=" ")

T = float(input())

SI = (P * R * T) / 100

print("Simple Interest is: " + str(SI))
```

```
Enter principal amount: 1000
Enter rate of interest (in %): 12
Enter time period (in years): 10
Simple Interest is: 1200.0
```

## Conclusion

It shows us how the program calculates gross salary and simple interest and how to effectively use input and output as well as type casting in python. It is used for understanding basic financial computations.

Aim – Use loops and mathematical concepts to produce a star pattern and factorial of entered number

Theory – It uses for loops in a fixed range to produce the desired output. The range function starts from 0 to the given number hence in the factorial program, we had to set the start to 1 as any number multiplied by zero is zero. In the star generation program, we used for loops to generate the empty spaces and the \*'s wherever required according to the pattern

A1.

n = 5

```
for i in range(n):
    for j in range(n - i - 1):
        print(' ', end='')
    for j in range(2 * i + 1):
        print('*', end='')
    print()
```

```
for i in range(n - 2, -1, -1):
    for j in range(n - i - 1):
        print(' ', end='')
    for j in range(2 * i + 1):
        print('*', end='')
    print()
```

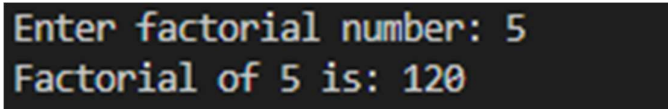


A2.

```
count = input("Enter factorial number: ")
```

```
factorial = 1
```

```
for i in range(1, int(count) + 1):  
    factorial = factorial * i  
print("Factorial of " + str(count) + " is: " + str(factorial))
```



```
Enter factorial number: 5  
Factorial of 5 is: 120
```

### Conclusion

It demonstrates the use of loops in python and how efficiently apply mathematical concepts and patterns in python.

# Built in Data Types LO1

30-1-26

Aim – Demonstrate the use of list, dictionaries and tuples and how to add remove and edit the data stored in them.

Theory –

The first program has the append and pop functions which add and remove data from a list respectively, they also structure it according to priority 1 to 3 as well as sorting the list. We use tuples to store the task and priority in the same index of the list.

Dictionaries are used in the next program to find the intersection, union and difference of the students appearing for neet, jee and cet. The functions are called by the sets and another set is given to the function to provide the required output.

The program is used to create update and manipulate a dictionary of student records including taking input and organizing their grades and attendance.

A1.

```
tasks = []
```

```
print("1-Add Task")
```

```
print("2-Remove Task")
```

```
print("3-Update Task")
```

```
print("4-Sort Tasks")
```

```
print("5-View All Tasks")
```

```
print("6-Exit")
```

```
while True:
```

```
    print("Enter your choice (1-6): ", end="")
```

```
    choice = int(input())
```

```
    if choice == 1:
```

```
        task_name = input("Enter task name: ")
```

```
        priority = int(input("Enter priority (1-Highest/2-Medium/3-Lowest): "))
```

```
        task = (task_name, priority)
```

```
        tasks.append(task)
```

```
        print(f"Task '{task_name}' added!")
```

```
    elif choice == 2:
```

```
        if not tasks:
```

```
print("No tasks to remove!")
```

```
else:
```

```
for i in range(len(tasks)):
```

```
    print(f'{i}. {tasks[i][0]} - {tasks[i][1]}')
```

```
print("Enter task number to remove: ", end="")
```

```
idx = int(input())
```

```
if 0 <= idx < len(tasks):
```

```
    removed = tasks.pop(idx)
```

```
    print(f'Task '{removed[0]}' removed!')
```

```
else:
```

```
    print("Invalid task number!")
```

```
elif choice == 3:
```

```
    if not tasks:
```

```
        print("No tasks to update!")
```

```
    else:
```

```
        for i in range(len(tasks)):
```

```
            print(f'{i}. {tasks[i][0]} - {tasks[i][1]}')
```

```
print("Enter task number to update: ", end="")
```

```
idx = int(input())
```

```
if 0 <= idx < len(tasks):
```

```
    new_name = input("Enter new task name: ")
```

```
    new_priority = int(input("Enter new priority (1-Highest/2-Medium/3-Lowest): "))
```

```
    tasks[idx] = (new_name, new_priority)
```

```
    print("Task updated!")
```

```
else:
```

```
    print("Invalid task number!")
```

```
elif choice == 4:
```

```
tasks.sort(key=lambda x: x[1])
```

```
print("Tasks sorted by priority!")
```

```
elif choice == 5:
```

if not tasks:

print("No tasks available!")

else:

print("\n--- All Tasks ---")

for i in range(len(tasks)):

print(f'{i}. {tasks[i][0]} - Priority: {tasks[i][1]}')

elif choice == 6:

print("Exit!")

break

else:

print("Invalid choice! Try again.")

```
1-Add Task
2-Remove Task
3-Update Task
4-Sort Tasks
5-View All Tasks
6-Exit
Enter your choice (1-6): 1
Enter task name: Test 1
Enter priority (1-Highest/2-Medium/3-Lowest): 1
Task 'Test 1' added!
Enter your choice (1-6): 5

--- All Tasks ---
0. Test 1 - Priority: 1
Enter your choice (1-6): 1
Enter task name: Test 2
Enter priority (1-Highest/2-Medium/3-Lowest): 3
Task 'Test 2' added!
Enter your choice (1-6): 5

--- All Tasks ---
0. Test 1 - Priority: 1
1. Test 2 - Priority: 3
Enter your choice (1-6): 2
0. Test 1 - 1
1. Test 2 - 3
Enter task number to remove: 0
Task 'Test 1' removed!
Enter your choice (1-6): 5

--- All Tasks ---
0. Test 2 - Priority: 3
Enter your choice (1-6): 1
Enter task name: Test 3
Enter priority (1-Highest/2-Medium/3-Lowest): 2
Task 'Test 3' added!
Enter your choice (1-6): 4
Tasks sorted by priority!
Enter your choice (1-6): 5

--- All Tasks ---
0. Test 3 - Priority: 2
1. Test 2 - Priority: 3
```

A2.

```
jee_students = {"Aarav", "Siddhant", "Vedant", "Chaitanya"}
cet_students = {"Siddhant", "Vedant", "Chaitanya", "Vikram"}
neet_students = {"Rohan", "Aarav", "Chaitanya", "Yuvraj"}

print(f'JEE Students: {jee_students}')
print(f'CET Students: {cet_students}')
print(f'NEET Students: {neet_students}')

(f'JEE union CET: {jee_students.union(cet_students)}')
print(f'JEE union NEET: {jee_students.union(neet_students)}')
print(f'CET union NEET: {cet_students.union(neet_students)}')

print(f'JEE intersection CET: {jee_students.intersection(cet_students)}')
print(f'JEE intersection NEET: {jee_students.intersection(neet_students)}')
print(f'CET intersection NEET: {cet_students.intersection(neet_students)}')

print(f'JEE - CET: {jee_students.difference(cet_students)}')
print(f'NEET - CET: {neet_students.difference(cet_students)}')
print(f'CET - JEE: {cet_students.difference(jee_students)}')
```

```
JEE Students: {'Aarav', 'Vedant', 'Siddhant', 'Chaitanya'}
CET Students: {'Vedant', 'Vikram', 'Siddhant', 'Chaitanya'}
NEET Students: {'Rohan', 'Aarav', 'Yuvraj', 'Chaitanya'}
JEE union NEET: {'Rohan', 'Aarav', 'Siddhant', 'Yuvraj', 'Chaitanya', 'Vedant'}
CET union NEET: {'Vikram', 'Rohan', 'Aarav', 'Siddhant', 'Yuvraj', 'Chaitanya', 'Vedant'}
JEE intersection CET: {'Vedant', 'Siddhant', 'Chaitanya'}
JEE intersection NEET: {'Aarav', 'Chaitanya'}
CET intersection NEET: {'Chaitanya'}
JEE - CET: {'Aarav'}
NEET - CET: {'Rohan', 'Aarav', 'Yuvraj'}
CET - JEE: {'Vikram'}
```

A3.

```
students = {}
```

```
while True:
```

```
    print("\n--- Student Records Management ---")
```

```
    print("1 - Add Student")
```



```
print("2 - Update Student")
print("3 - View All Students")
print("4 - View Specific Student")
print("5 - Delete Student")
print("6 - Exit")
```

```
choice = input("Enter your choice (1-6): ")
```

```
if choice == "1":
```

```
    roll_no = input("Enter Roll Number: ")
```

```
    if roll_no in students:
```

```
        print("Student already exists!")
```

```
    else:
```

```
        name = input("Enter Name: ")
```

```
        math_grade = float(input("Enter Math grade: "))
```

```
        english_grade = float(input("Enter English grade: "))
```

```
        science_grade = float(input("Enter Science grade: "))
```

```
        attendance = float(input("Enter Attendance percentage: "))
```

```
        grades = {
```

```
            "Math": math_grade,
```

```
            "English": english_grade,
```

```
            "Science": science_grade
```

```
        }
```

```
        students[roll_no] = {
```

```
            "name": name,
```

```
            "grades": grades,
```

```
            "attendance": attendance
```

```
        }
```

```
        print("Student added successfully!")
```

```

elif choice == "2":

    roll_no = input("Enter Roll Number to update: ")

    if roll_no not in students:

        print("Student not found!")
    else:

        print("\nWhat do you want to update?")

        print("1. Grades")
        print("2. Attendance")

        update_choice = input("Enter choice (1-2): ")

        if update_choice == "1":

            subject = input("Enter subject (Math/English/Science): ")
            new_grade = float(input(f"Enter new {subject} grade: "))
            students[roll_no]["grades"][subject] = new_grade
            print("Grade updated!")

        elif update_choice == "2":

            new_attendance = float(input("Enter new attendance: "))
            students[roll_no]["attendance"] = new_attendance
            print("Attendance updated!")

elif choice == "3":

    if not students:

        print("No students in records!")
    else:

        print("\n--- All Students ---")

        for roll, details in students.items():

            print(f"\nRoll No: {roll}")
            print(f'Name: {details["name"]}')
            print(f'Grades: {details["grades"]}')

```

```
print(f'Attendance: {details['attendance']}%')
```

```
elif choice == "4":
```

```
roll_no = input("Enter Roll Number: ")
```

```
if roll_no not in students:
```

```
    print("Student not found!")
```

```
else:
```

```
    details = students[roll_no]
```

```
    print(f'\n--- Student Details ---')
```

```
    print(f'Roll No: {roll_no}')
```

```
    print(f'Name: {details['name']}')
```

```
    print(f'Grades: {details['grades']}')
```

```
    print(f'Attendance: {details['attendance']}%')
```

```
elif choice == "5":
```

```
roll_no = input("Enter Roll Number to delete: ")
```

```
if roll_no not in students:
```

```
    print("Student not found!")
```

```
else:
```

```
    del students[roll_no]
```

```
    print("Student deleted successfully!")
```

```
elif choice == "6":
```

```
    print("Exiting...")
```

```
    break
```

```
else:
```

```
    print("Invalid choice! Please try again.")
```

```

--- Student Records Management ---
1 - Add Student
2 - Update Student
3 - View All Students
4 - View Specific Student
5 - Delete Student
6 - Exit
Enter your choice (1-6): 1
Enter Roll Number: 1
Enter Name: Vedant
Enter Math grade: 50
Enter English grade: 55
Enter Science grade: 56
Enter Attendance percentage: 80
Student added successfully!

--- Student Records Management ---
1 - Add Student
2 - Update Student
3 - View All Students
4 - View Specific Student
5 - Delete Student
6 - Exit
Enter your choice (1-6): 1
Enter Roll Number: 5
Enter Name: Aarav
Enter Math grade: 40
Enter English grade: 70
Enter Science grade: 70
Enter Attendance percentage: 20
Student added successfully!

--- Student Records Management ---
1 - Add Student
2 - Update Student
3 - View All Students
4 - View Specific Student
5 - Delete Student
6 - Exit
Enter your choice (1-6): 3

--- All Students ---

Roll No: 1
Name: Vedant
Grades: {'Math': 50.0, 'English': 55.0, 'Science': 56.0}
Attendance: 80.0%

Roll No: 5
Name: Aarav
Grades: {'Math': 40.0, 'English': 70.0, 'Science': 70.0}
Attendance: 20.0%

--- Student Records Management ---
1 - Add Student
2 - Update Student
3 - View All Students
4 - View Specific Student
5 - Delete Student
6 - Exit
Enter your choice (1-6): 2
Enter Roll Number to update: 1

What do you want to update?
1. Grades
2. Attendance
Enter choice (1-2): 2
Enter new attendance: 40
Attendance updated!

```

```

--- Student Records Management ---
1 - Add Student
2 - Update Student
3 - View All Students
4 - View Specific Student
5 - Delete Student
6 - Exit
Enter your choice (1-6): 4
Enter Roll Number: 1

--- Student Details ---
Roll No: 1
Name: Vedant
Grades: {'Math': 50.0, 'English': 55.0, 'Science': 56.0}
Attendance: 40.0%

--- Student Records Management ---
1 - Add Student
2 - Update Student
3 - View All Students
4 - View Specific Student
5 - Delete Student
6 - Exit
Enter your choice (1-6): 5
Enter Roll Number to delete: 5
Student deleted successfully!

--- Student Records Management ---
1 - Add Student
2 - Update Student
3 - View All Students
4 - View Specific Student
5 - Delete Student
6 - Exit
Enter your choice (1-6): 3

--- All Students ---

Roll No: 1
Name: Vedant
Grades: {'Math': 50.0, 'English': 55.0, 'Science': 56.0}
Attendance: 40.0%

```

## Conclusion

It demonstrates the practical use of lists, sets and dictionaries in python. It creates a menu driven program using for loops, logical flow and user interaction. It shows is how to efficiently organize data and exam records. It gives us practical problem solving using pythons core concepts.

# File Handling LO3

6-2-26

Aim – Demonstrate file handling in python.

Theory – File handling in Python allows programs to store and retrieve data permanently using files. It supports operations such as opening, reading, writing, and closing files. Functions like open(), read(), readlines(), and context managers (with) ensure efficient data processing and safe resource management.

A1.

```
file_name = "test.txt"
```

```
word_length = int(input("Enter desired word length: "))
```

```
file = open(file_name, 'r')
```

```
content = file.read()
```

```
file.close()
```

```
words = content.split()
```

```
print(f"Words with length {word_length}:")
```

```
for word in words:
```

```
    cleaned_word = ""
```

```
    for char in word:
```

```
        if char.isalpha():
```

```
            cleaned_word += char
```

```
    if len(cleaned_word) == word_length:
```

```
        print(cleaned_word)
```

```
C:\Users\tarun\Downloads\TSEC-SEM2-PYTHON\Assignment 4>python a1.py
Enter desired word length: 5
Words with length 5:
Delhi
```

A2.

```
with open('test.txt', 'r') as file:
```

```
    cities = file.readlines()
```

```
cities = [city.strip() for city in cities]
```

```
cities.sort()
```

```
for city in cities:
```

```
    print(city)
```

```
C:\Users\tarun\Downloads\TSEC-SEM2-PYTHON\Assignment 4>python a2.py
```

```
Ahmedabad
```

```
Bangalore
```

```
Chennai
```

```
Delhi
```

```
Hyderabad
```

```
Jaipur
```

```
Kolkata
```

```
Lucknow
```

```
Mumbai
```

```
Pune
```

## Conclusion

File handling is essential for working with external data in Python programs. Using proper methods and context managers helps prevent data loss and memory leaks. It makes programs more reliable, organized, and suitable for real-world applications involving persistent data storage.

# Object Oriented Programming LO4

13-2-26

Aim – Design a system using classes for vehicles, rental agencies, and rental transactions. Implement methods to handle vehicle availability, rental periods, pricing, and customer bookings.

Theory –

Object-Oriented Programming in Python organizes code using classes and objects. A class defines attributes and methods, while objects are instances of classes. Inheritance allows new classes to reuse properties from parent classes. Method overriding enables subclasses to modify behavior. This approach improves modularity, reusability, and structure of programs.

A1.

```
from datetime import datetime, timedelta
```

```
class Vehicle:
```

```
    def __init__(self, id, name, daily_rate):
```

```
        self.id = id
```

```
        self.name = name
```

```
        self.daily_rate = daily_rate
```

```
        self.available = True
```

```
class RentalAgency:
```

```
    def __init__(self, name):
```

```
        self.name = name
```

```
        self.vehicles = []
```

```
        self.transactions = []
```

```
    def add_vehicle(self, vehicle):
```

```
        self.vehicles.append(vehicle)
```

```
    def book_vehicle(self, vehicle_id, customer, days):
```

```
        vehicle = next((v for v in self.vehicles if v.id == vehicle_id), None)
```

```
        if not vehicle or not vehicle.available:
```

```
            return None
```

```
cost = vehicle.daily_rate * days
rental = RentalTransaction(vehicle, customer, days, cost)
vehicle.available = False
self.transactions.append(rental)

return rental
```

```
def return_vehicle(self, vehicle_id):
    vehicle = next((v for v in self.vehicles if v.id == vehicle_id), None)
    if vehicle:
        vehicle.available = True
```

```
class RentalTransaction:
```

```
    def __init__(self, vehicle, customer, days, cost):
        self.vehicle = vehicle
        self.customer = customer
        self.days = days
        self.cost = cost
        self.start_date = datetime.now()
        self.end_date = self.start_date + timedelta(days=days)
```

```
    def __str__(self):
        return f'{self.customer} rented {self.vehicle.name} for {self.days} days - Cost: {self.cost}'
```

```
agency = RentalAgency("QuickRent")
agency.add_vehicle(Vehicle(1, "Maruti", 50))
agency.add_vehicle(Vehicle(2, "Honda", 60))
```

```
rental = agency.book_vehicle(1, "Aarav", 5)
print(rental)
```

```
agency.return_vehicle(1)
print(f'Vehicle 1 available: {agency.vehicles[0].available}')
```



```
Aarav rented Maruti for 5 days - Cost: 250  
Vehicle 1 available: True
```

Conclusion

OOP is used to encapsulate various functions and provide data abstraction.

# Exception Handling LO3

20-2-26

Aim – Write a program to take 2 numbers as input and perform division while demonstrating exception handling with zero and wrong inputs.

Demonstrate the use of a python debugger with intentional errors on a sample debugger

Theory – Exception handling in Python is used to manage runtime errors without stopping program execution. The try block contains code that may cause errors, while the except block handles specific exceptions like ZeroDivisionError, ValueError, IndexError, KeyError, and FileNotFoundError. Custom exceptions allow programmers to define and handle application-specific errors.

A1.

try:

```
num1 = input("Enter the first number: ")
num2 = input("Enter the second number: ")
num1 = float(num1)
num2 = float(num2)
result = num1 / num2
print(f"Result of {num1} / {num2} = {result}")
```

except ZeroDivisionError:

```
print("Error: Cannot divide by zero! Please enter a non-zero divisor.")
```

except ValueError:

```
print("Error: Invalid input! Please enter valid numeric values.")
```

except Exception as e:

```
print(f"An unexpected error occurred: {e}")
```

else:

```
print("Division completed successfully!")
```

finally:

```
print("Program execution finished.")
```

```
Enter the first number: 10
Enter the second number: 0
Error: Cannot divide by zero! Please enter a non-zero divisor.
Program execution finished.
```

```
Enter the first number: 10
Enter the second number: 2
Result of 10.0 / 2.0 = 5.0
Division completed successfully!
Program execution finished.
```

```
Enter the first number: AA
Enter the second number: 12
Error: Invalid input! Please enter valid numeric values.
Program execution finished.
```

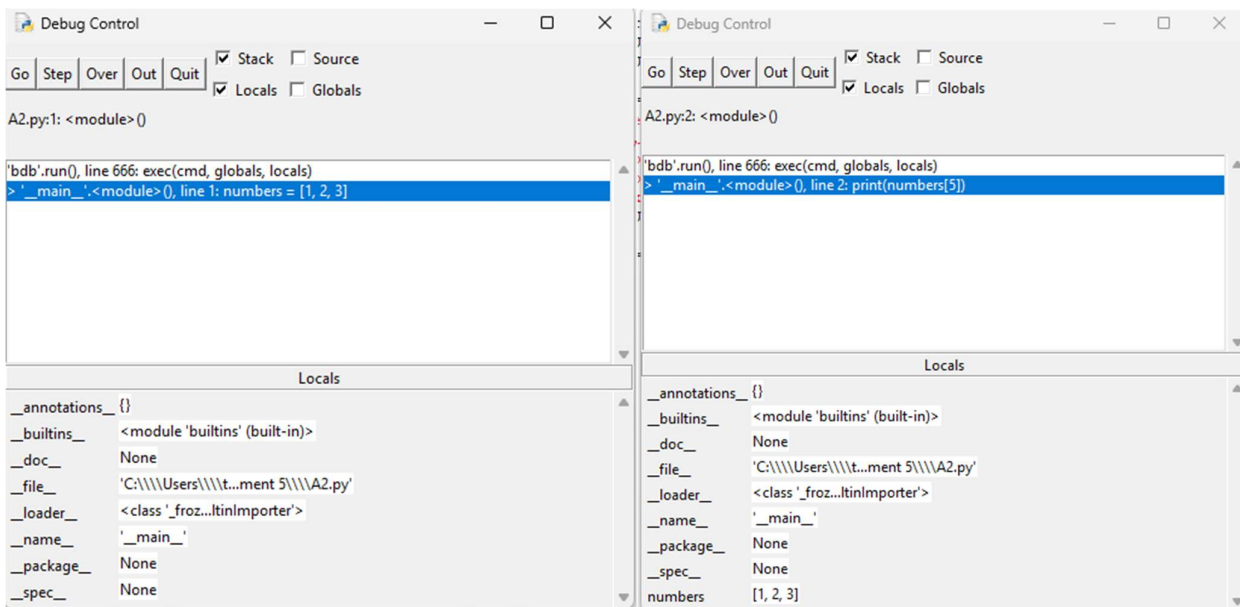
A2.

```
numbers = [1, 2, 3]
```

```
print(numbers[5])
```

```
result = 10/0
```

```
print(result)
```



## Conclusion

This program demonstrates exception handling and its various use cases. Exception handling improves program reliability by preventing crashes and allowing developers to manage different runtime errors in a controlled way.

## Arrays LO6

20-2-26

Aim – Write a program to create a 1D, 2D and 3D NumPy array. Perform basic operations like reshaping, slicing, and indexing. Calculate the dot product and cross product of 2 vectors.  
Develop a python script to create 2 arrays of the same shape and perform element wise addition, subtraction, multiplication, and division.

Theory - NumPy is a powerful Python library used for numerical computing. It provides support for multidimensional arrays (1D, 2D, 3D) and efficient operations on them. It allows reshaping arrays, slicing, and indexing to access or modify elements easily. NumPy also supports mathematical operations like addition, subtraction, multiplication, and division, as well as advanced computations such as dot product and cross product. It is widely used for scientific computing, data analysis, and machine learning due to its speed and efficiency.

A1.

```
import numpy as np
```

```
array_1d = np.array([1, 2, 3, 4, 5])
```

```
print("1D Array:")
```

```
print(array_1d)
```

```
array_2d = np.array([[1, 2, 3], [4, 5, 6], [7, 8, 9]])
```

```
print("\n2D Array:")
```

```
print(array_2d)
```

```
array_3d = np.array([[[1, 2], [3, 4]], [[5, 6], [7, 8]]])
```

```
print("\n3D Array:")
```

```
print(array_3d)
```

```
print("\n--- Reshaping ---")
```

```
reshaped_1d = array_1d.reshape(5, 1)
```

```
print("1D reshaped to 5x1:")
```

```
print(reshaped_1d)
```

```
reshaped_2d = array_2d.reshape(1, 9)
```

```
print("\n2D reshaped to 1x9:")
```

```
print(reshaped_2d)
```

```
print("\n--- Slicing ---")  
print("1D array slice [1:4]:", array_1d[1:4])  
print("2D array slice [0:2, 1:3]:")  
print(array_2d[0:2, 1:3])  
print("3D array slice [0, 1, :]:")  
print(array_3d[0, 1, :])
```

```
print("\n--- Indexing ---")  
print("1D array index [2]:", array_1d[2])  
print("2D array index [1, 2]:", array_2d[1, 2])  
print("3D array index [1, 0, 1]:", array_3d[1, 0, 1])
```

```
print("\n--- Dot Product ---")  
vector1 = np.array([1, 2, 3])  
vector2 = np.array([4, 5, 6])  
dot_product = np.dot(vector1, vector2)  
print(f"Vector 1: {vector1}")  
print(f"Vector 2: {vector2}")  
print(f"Dot Product: {dot_product}")
```

```
print("\n--- Cross Product ---")  
cross_product = np.cross(vector1, vector2)  
print(f"Cross Product: {cross_product}")
```

```

1D Array:
[1 2 3 4 5]

2D Array:
[[1 2 3]
 [4 5 6]
 [7 8 9]]

3D Array:
[[[1 2]
  [3 4]]
 [[5 6]
  [7 8]]]

--- Reshaping ---
1D reshaped to 5x1:
[[1]
 [2]
 [3]
 [4]
 [5]]

2D reshaped to 1x9:
[[1 2 3 4 5 6 7 8 9]]

--- Slicing ---
1D array slice [1:4]: [2 3 4]
2D array slice [0:2, 1:3]:
[[2 3]
 [5 6]]
3D array slice [0, 1, :]:
[3 4]

--- Indexing ---
1D array index [2]: 3
2D array index [1, 2]: 6
3D array index [1, 0, 1]: 6

--- Dot Product ---
Vector 1: [1 2 3]
Vector 2: [4 5 6]
Dot Product: 32

--- Cross Product ---
Cross Product: [-3  6 -3]

```

A2.

```
import numpy as np
```

```
array1 = np.array([10, 20, 30, 40])
```

```
array2 = np.array([2, 4, 5, 8])
```

```
addition = array1 + array2
```

```
print("Addition:", addition)
```

```
subtraction = array1 - array2
```

```
print("Subtraction:", subtraction)
```

```
multiplication = array1 * array2
```

```
print("Multiplication:", multiplication)
```

```
division = array1 / array2
```

```
print("Division:", division)
```

```

Addition: [12 24 35 48]
Subtraction: [ 8 16 25 32]
Multiplication: [ 20  80 150 320]
Division: [5.  5.  6.  5.]

```

## Conclusion

This program demonstrates the numpy library and its various use cases.

Aim – Develop a Python GUI application to create Student Form.

Theory –

This Python program creates a graphical user interface (GUI) student form using the Tkinter library. It collects user inputs such as name, age, gender, selected subjects, and grade. Various widgets like labels, entry fields, radio buttons, checkboxes, and a combobox are used to design the interface. The submit function retrieves the entered data and displays it in a text box within the same window. Boolean variables handle multiple subject selections, while StringVar manages gender input. The place() geometry manager is used for positioning elements. This program demonstrates event-driven programming, user input handling, and basic GUI design in Python.

A1.

```
import tkinter as tk
```

```
from tkinter import ttk
```

```
def submit_form():
```

```
    name = name_entry.get()
```

```
    age = age_entry.get()
```

```
    gender = gender_var.get()
```

```
    subjects = []
```

```
    if math_var.get():
```

```
        subjects.append("Math")
```

```
    if sci_var.get():
```

```
        subjects.append("Sci")
```

```
    if eng_var.get():
```

```
        subjects.append("Eng")
```

```
    grade = grade_combobox.get()
```

```
    textbox.insert(tk.END, f"Name: {name}\n")
```

```
    textbox.insert(tk.END, f"Age: {age}\n")
```

```
    textbox.insert(tk.END, f"Gender: {gender}\n")
```

```
    textbox.insert(tk.END, f"Subjects: {' '.join(subjects)}\n")
```

```
    textbox.insert(tk.END, f"Grade: {grade}\n")
```

```
    textbox.insert(tk.END, f"~"*40+"\n")
```



```
root = tk.Tk()
root.title("Student form")
root.geometry("400x500")
tk.Label(root,text="Name:").place(x=20,y=20)
name_entry = tk.Entry(root, width =30)
name_entry.place(x=120,y=20)

tk.Label(root, text = "Age:").place(x=20,y=60)
age_entry = tk.Entry(root, width =30)
age_entry.place(x=120,y=60)

tk.Label(root, text = "Gender:").place(x=20, y =100)
gender_var = tk.StringVar(value = "None")
tk.Radiobutton(root, text = "Male", variable = gender_var, value ="Male").place(x=120,y=100)
tk.Radiobutton(root, text = "Female", variable = gender_var, value ="Female").place(x=180,y=100)

tk.Label(root,text="Subjects:").place(x=20,y=140)
math_var=tk.BooleanVar()
sci_var=tk.BooleanVar()
eng_var=tk.BooleanVar()
tk.Checkbutton(root,text="Math", variable = math_var).place(x=120,y=140)
tk.Checkbutton(root,text="Eng",variable = eng_var).place(x=120,y=170)
tk.Checkbutton(root,text="Sci",variable = sci_var).place(x=120,y=200)

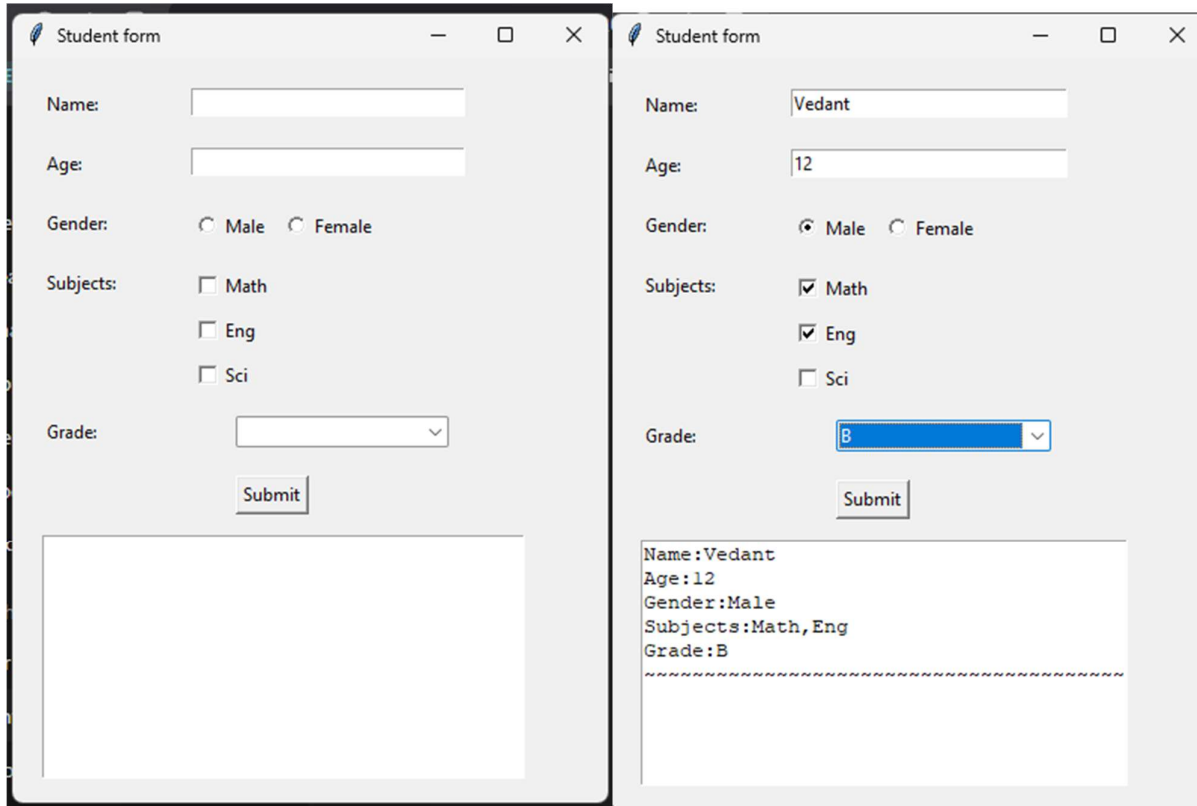
tk.Label(root, text="Grade:").place(x=20,y=240)
grade_combobox = ttk.Combobox(root, values = ["A","B","C","D","E"],state = "readonly")
grade_combobox.place(x=150,y=240)

submit_button = tk.Button(root, text = "Submit", command = submit_form)
submit_button.place(x=150, y=280)
```

```
textbox = tk.Text(root, width = 40, height = 10)
```

```
textbox.place(x=20,y=320)
```

```
root.mainloop()
```



## Conclusion

This application shows how Tkinter can be used to create interactive forms. It highlights input collection, widget usage, and data display. Such simple GUI programs are useful for beginners to understand user interaction and desktop application development basics.

Aim – Using the Iris Data (<https://www.kaggle.com/datasets/saurabh00007/iris.csv>), perform the following tasks:

- i. Read the first 8 rows of the dataset.
- ii. Display the column names of the Iris dataset.
- iii. Fill any missing data with the mean value of the respective column.
- iv. Remove rows that contain any missing values.
- v. Calculate and display the mean, minimum, and maximum values of the Sepal length column.

LO6

17 Using the Cars Data (<https://www.kaggle.com/datasets/nameeerafatima/toyotacsv>) perform the following tasks:

- i. Create a scatter plot between the Age and Price of the cars to illustrate how the price decreases as the age of the car increases.
- ii. Generate a histogram to show the frequency distribution of kilometres driven by the cars.
- iii. Produce a bar plot to display the distribution of cars by fuel type.
- iv. Create a pie chart to represent the percentage distribution of cars based on fuel types.
- v. Draw a box plot to visualize the distribution of car prices across different fuel types.

Theory –

This program demonstrates basic data analysis and visualization using Python libraries like Pandas and Matplotlib. First, the Iris dataset is loaded to explore data preprocessing techniques such as handling missing values. The dataset is inspected by displaying the first few rows and column names. Missing values are managed using two approaches: filling numerical columns with mean values and removing rows containing null values. Statistical analysis is then performed on the “SepalLengthCm” column to calculate mean, minimum, and maximum values.

In the second part, the Toyota dataset is used for visual analysis. Various plots are created to understand relationships and distributions within the data. A scatter plot shows how car price varies with age, while a histogram displays the distribution of kilometers driven. Bar and pie charts illustrate the frequency and percentage of different fuel types. Finally, a boxplot compares price distributions across fuel types, helping identify variations and outliers effectively.

A1.

```
import pandas as pd
```

```
df = pd.read_csv('iris.csv')
```

```

print("First 8 rows of the dataset:")
print(df.head(8))

print("\nColumn names:")
print(df.columns.tolist())

df_filled = df.fillna(df.mean(numeric_only=True))

df_cleaned = df.dropna()

```

```

sepal_length = df_cleaned['SepalLengthCm']

```

```

print("\nSepal Length Statistics:")
print(f"Mean: {sepal_length.mean()}")
print(f"Minimum: {sepal_length.min()}")
print(f"Maximum: {sepal_length.max()}")

```

```

First 8 rows of the dataset:
   Id  SepalLengthCm  SepalWidthCm  PetalLengthCm  PetalWidthCm  Species
0   1             5.1           3.5           1.4           0.2  Iris-setosa
1   2             4.9           3.0           1.4           0.2  Iris-setosa
2   3             4.7           3.2           1.3           0.2  Iris-setosa
3   4             4.6           3.1           1.5           0.2  Iris-setosa
4   5             5.0           3.6           1.4           0.2  Iris-setosa
5   6             5.4           3.9           1.7           0.4  Iris-setosa
6   7             4.6           3.4           1.4           0.3  Iris-setosa
7   8             5.0           3.4           1.5           0.2  Iris-setosa

Column names:
['Id', 'SepalLengthCm', 'SepalWidthCm', 'PetalLengthCm', 'PetalWidthCm', 'Species']

Sepal Length Statistics:
Mean: 5.843333333333334
Minimum: 4.3
Maximum: 7.9

```

A2.

```

import pandas as pd

```

```

import matplotlib.pyplot as plt

```

```

df = pd.read_csv('Toyota.csv')

```

```
plt.figure(figsize=(10, 6))
plt.scatter(df['Age'], df['Price'], alpha=0.6, color='blue')
plt.xlabel('Age (years)')
plt.ylabel('Price')
plt.title('Car Price vs Age')
plt.grid(True, alpha=0.3)
plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(10, 6))
plt.hist(df['KM'], bins=30, color='green', edgecolor='black')
plt.xlabel('Kilometres Driven')
plt.ylabel('Frequency')
plt.title('Distribution of Kilometres Driven')
plt.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(10, 6))
fuel_counts = df['FuelType'].value_counts()
plt.bar(fuel_counts.index, fuel_counts.values, color='orange', edgecolor='black')
plt.xlabel('Fuel Type')
plt.ylabel('Number of Cars')
plt.title('Distribution of Cars by Fuel Type')
plt.grid(True, alpha=0.3, axis='y')
plt.tight_layout()
plt.show()
```

```
plt.figure(figsize=(8, 8))
fuel_counts = df['FuelType'].value_counts()
plt.pie(fuel_counts.values, labels=fuel_counts.index, autopct='%1.1f%%', startangle=90)
plt.title('Percentage Distribution of Cars by Fuel Type')
```

```
plt.tight_layout()
```

```
plt.show()
```

```
plt.figure(figsize=(10, 6))
```

```
df.boxplot(column='Price', by='FuelType', figsize=(10, 6))
```

```
plt.xlabel('Fuel Type')
```

```
plt.ylabel('Price')
```

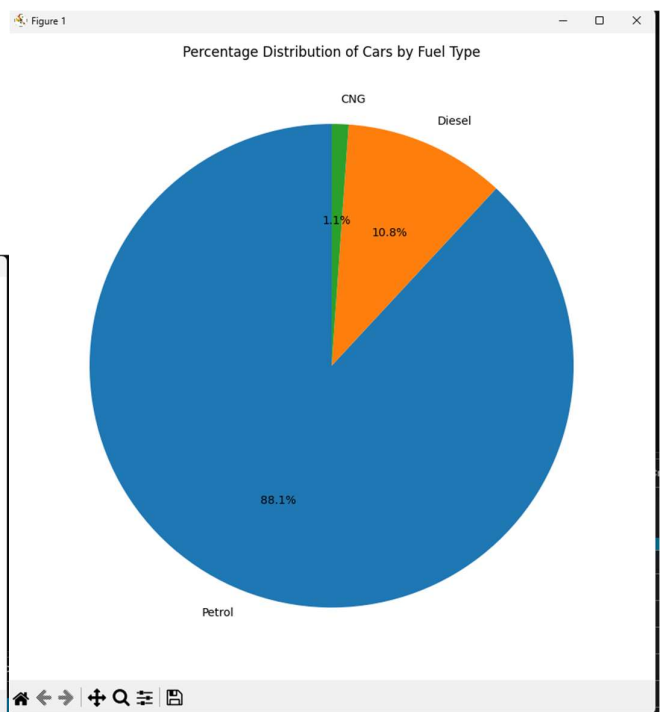
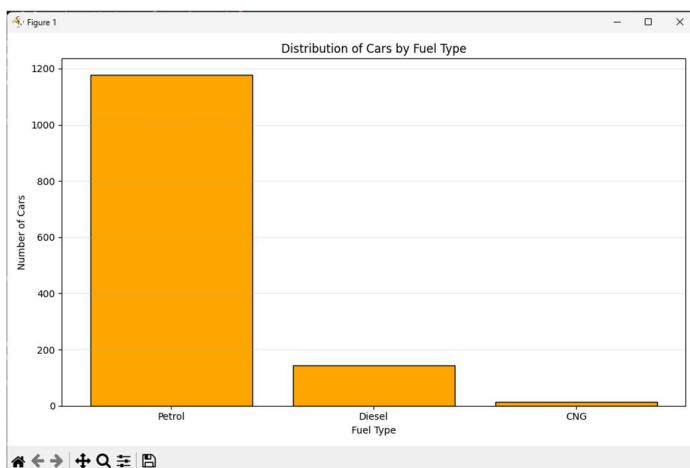
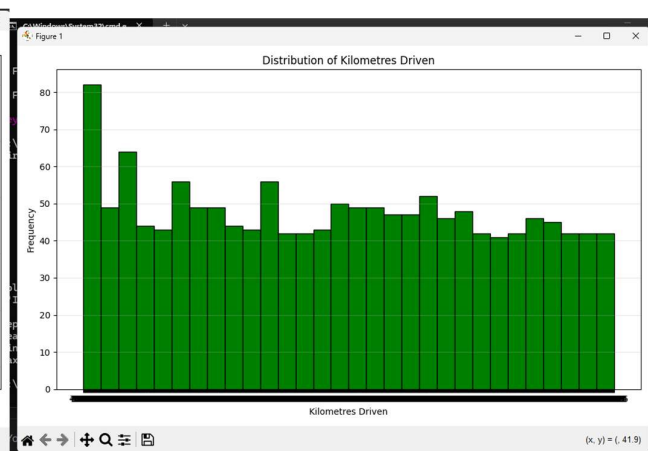
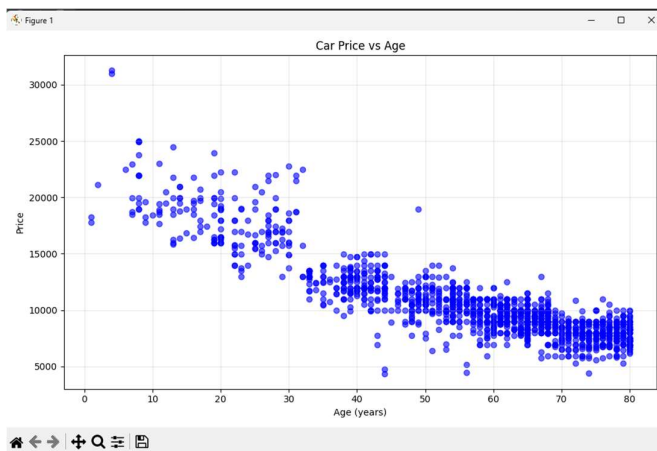
```
plt.title('Distribution of Car Prices by Fuel Type')
```

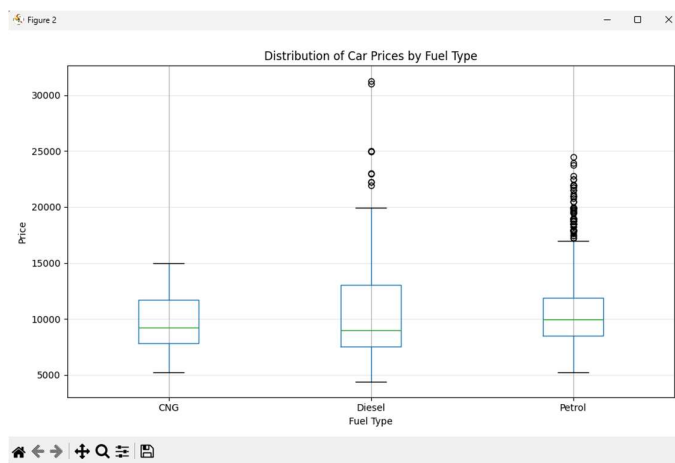
```
plt.suptitle("") # Remove the default title
```

```
plt.grid(True, alpha=0.3, axis='y')
```

```
plt.tight_layout()
```

```
plt.show()
```





## Conclusion

The program effectively combines data cleaning, statistical analysis, and visualization techniques. It highlights how datasets can be explored and interpreted using Python. Visual tools make patterns and relationships clearer, while preprocessing ensures accuracy, making the overall analysis more meaningful and reliable for decision-making.

## Pattern Matching LO5

23-3-26

Aim – Write a Python script that prompts the user to enter E-mail and Password. Use regular expressions to validate the E-mail and Password based on these criteria: Password length should contain at least 8 characters long and maximum 12. It should have at least one uppercase letter, one lowercase letter, one digit, and one special character. E-mail should have all small letters.

Theory – This Python program validates an email and password using regular expressions from the re module. The user is prompted to input an email and password, which are then checked against specific patterns. The email pattern ensures lowercase letters, digits, and valid symbols, followed by a proper domain format. The password pattern enforces security rules such as length between 8 to 12 characters, and inclusion of uppercase letters, lowercase letters, digits, and special characters. The re.match() function is used to compare inputs with patterns. Based on validation results, appropriate messages are displayed, demonstrating input validation and basic security checks.

A1.

```
import re
```

```
# Prompt user for email
```

```
email = input("Enter E-mail: ")
```

```
# Prompt user for password
```

```
password = input("Enter Password: ")
```

```
# Email validation - all small letters
```

```
email_pattern = r'^[a-z0-9._%+-]+@[a-z0-9.-]+\.[a-z]{2,}$'
```

```
email_valid = re.match(email_pattern, email)
```

```
# Password validation
```

```
# At least 8 characters, max 12
```

```
# At least one uppercase, one lowercase, one digit, one special character
```

```
password_pattern = r'^(?=.*[A-Z])(?=.*[a-z])(?=.*\d)(?=.*[@$!%*?&])[A-Za-z\d@$!%*?&]{8,12}$'
```

```
password_valid = re.match(password_pattern, password)
```

```
# Display results
```

```
if email_valid:
```



```
print("✓ E-mail is valid")
```

else:

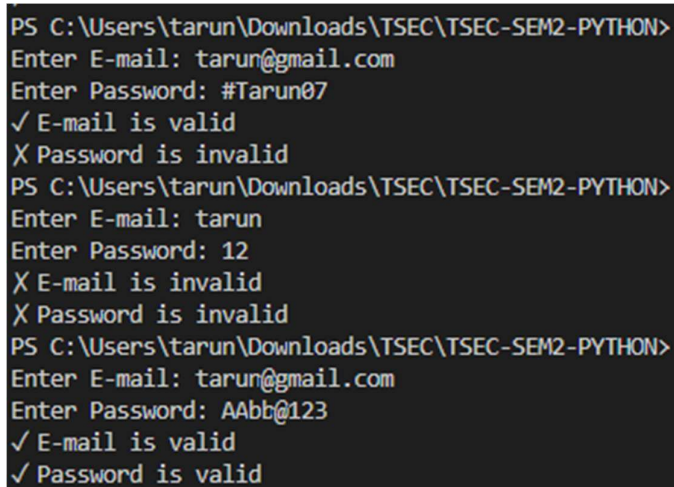
```
print("X E-mail is invalid")
```

if password\_valid:

```
print("✓ Password is valid")
```

else:

```
print("X Password is invalid")
```



```
PS C:\Users\tarun\Downloads\TSEC\TSEC-SEM2-PYTHON>
Enter E-mail: tarun@gmail.com
Enter Password: #Tarun07
✓ E-mail is valid
X Password is invalid
PS C:\Users\tarun\Downloads\TSEC\TSEC-SEM2-PYTHON>
Enter E-mail: tarun
Enter Password: 12
X E-mail is invalid
X Password is invalid
PS C:\Users\tarun\Downloads\TSEC\TSEC-SEM2-PYTHON>
Enter E-mail: tarun@gmail.com
Enter Password: AAbb@123
✓ E-mail is valid
✓ Password is valid
```

## Conclusion

This program demonstrates simple validation using regex, ensuring correct email format and strong passwords for secure user input handling.